Delegate - C++17

Generated by Doxygen 1.16.1

1 Namespace Index 1
1.1 Namespace List . . . 1

2 Hierarchical Index 3
2.1 Class Hierarchy . . . 3

3 Class Index 5
3.1 Class List . . . 5

4 File Index 7
4.1 File List . . . 7

5 Namespace Documentation 9
5.1 bricxx Namespace Reference . . . 9

6 Class Documentation 11
6.1 bricxx::BadDelegateCall Class Reference . . . 11
6.1.1 Detailed Description . . . 11
6.1.2 Member Function Documentation . . . 11
6.1.2.1 what() . . . 11
6.2 bricxx::Delegate< TSignature > Class Template Reference . . . 12
6.2.1 Detailed Description . . . 12
6.3 bricxx::Delegate< TRetVal(TParams...) const > Class Template Reference . . . 14
6.3.1 Detailed Description . . . 15
6.3.2 Constructor & Destructor Documentation . . . 15
6.3.2.1 Delegate() [1/2] . . . 15
6.3.2.2 Delegate() [2/2] . . . 15
6.3.2.3 ~Delegate() . . . 15
6.3.3 Member Function Documentation . . . 15
6.3.3.1 operator=() . . . 15
6.3.3.2 operator==() . . . 15
6.3.3.3 operator"!=() . . . 16
6.3.3.4 Bind() [1/2] . . . 16
6.3.3.5 Bind() [2/2] . . . 16
6.3.3.6 IsBindedTo() [1/2] . . . 16
6.3.3.7 IsBindedTo() [2/2] . . . 16
6.3.3.8 operator()() . . . 17
6.4 bricxx::Delegate< TRetVal(TParams...) const noexcept > Class Template Reference . . . 17
6.4.1 Detailed Description . . . 18
6.4.2 Constructor & Destructor Documentation . . . 18
6.4.2.1 Delegate() [1/2] . . . 18
6.4.2.2 Delegate() [2/2] . . . 19
6.4.2.3 ~Delegate() . . . 19
6.4.3 Member Function Documentation . . . 19

6.4.3.1 operator=() . . . 19
6.4.3.2 operator==() . . . 19
6.4.3.3 operator"!=() . . . 19
6.4.3.4 Bind() [1/2] . . . 19
6.4.3.5 Bind() [2/2] . . . 20
6.4.3.6 IsBindedTo() [1/2] . . . 20
6.4.3.7 IsBindedTo() [2/2] . . . 20
6.4.3.8 operator()() . . . 20
6.5 bricxx::Delegate< TRetVal(TParams...) noexcept > Class Template Reference . . . 21
6.5.1 Detailed Description . . . 22
6.5.2 Constructor & Destructor Documentation . . . 22
6.5.2.1 Delegate() [1/2] . . . 22
6.5.2.2 Delegate() [2/2] . . . 22
6.5.2.3 ~Delegate() . . . 22
6.5.3 Member Function Documentation . . . 22
6.5.3.1 operator=() . . . 22
6.5.3.2 operator==() . . . 22
6.5.3.3 operator"!=() . . . 23
6.5.3.4 Bind() [1/3] . . . 23
6.5.3.5 Bind() [2/3] . . . 23
6.5.3.6 Bind() [3/3] . . . 23
6.5.3.7 IsBindedTo() [1/3] . . . 24
6.5.3.8 IsBindedTo() [2/3] . . . 24
6.5.3.9 IsBindedTo() [3/3] . . . 24
6.5.3.10 operator()() . . . 24
6.6 bricxx::Delegate< TRetVal(TParams...)> Class Template Reference . . . 25
6.6.1 Detailed Description . . . 26
6.6.2 Constructor & Destructor Documentation . . . 26
6.6.2.1 Delegate() [1/2] . . . 26
6.6.2.2 Delegate() [2/2] . . . 26
6.6.2.3 ~Delegate() . . . 26
6.6.3 Member Function Documentation . . . 26
6.6.3.1 operator=() . . . 26
6.6.3.2 operator==() . . . 26
6.6.3.3 operator"!=() . . . 27
6.6.3.4 Bind() [1/3] . . . 27
6.6.3.5 Bind() [2/3] . . . 27
6.6.3.6 Bind() [3/3] . . . 27
6.6.3.7 IsBindedTo() [1/3] . . . 28
6.6.3.8 IsBindedTo() [2/3] . . . 28
6.6.3.9 IsBindedTo() [3/3] . . . 28
6.6.3.10 operator()() . . . 28

6.7 bricxx::DelegateBase< TRetVal, TParams > Class Template Reference . . . 29
6.7.1 Detailed Description . . . 30
6.7.2 Member Typedef Documentation . . . 30
6.7.2.1 ErasedObjectType . . . 30
6.7.2.2 StubFnPtr . . . 30
6.7.3 Constructor & Destructor Documentation . . . 30
6.7.3.1 DelegateBase() [1/2] . . . 30
6.7.3.2 DelegateBase() [2/2] . . . 30
6.7.3.3 ~DelegateBase() . . . 30
6.7.4 Member Function Documentation . . . 31
6.7.4.1 IsBinded() . . . 31
6.7.4.2 operator bool() . . . 31
6.7.4.3 Unbind() . . . 31
6.7.4.4 Reset() . . . 31
6.7.4.5 operator=() . . . 31
6.7.4.6 operator==() . . . 31
6.7.4.7 operator"!=() . . . 31
6.7.4.8 NullStub() . . . 32
6.7.5 Member Data Documentation . . . 32
6.7.5.1 pStub_ . . . 32
6.7.5.2 pTypeErasedObject_ . . . 32

7 File Documentation **33**
7.1 delegate.hpp File Reference . . . 33
7.1.1 Macro Definition Documentation . . . 34
7.1.1.1 BRICXX_DELEGATE_API . . . 34
7.2 delegate.hpp . . . 34

Index **47**

Chapter 1

Namespace Index

1.1 Namespace List

Here is a list of all namespaces with brief descriptions:

bricxx 9

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

bricxx::Delegate< TSignature > 12
bricxx::DelegateBase< TRetVal, TParams > 29
bricxx::DelegateBase< TRetVal, TParams... > 29
  bricxx::Delegate< TRetVal(TParams...) const > 14
  bricxx::Delegate< TRetVal(TParams...) const noexcept > 17
  bricxx::Delegate< TRetVal(TParams...) noexcept > 21
  bricxx::Delegate< TRetVal(TParams...)> 25
std::exception
  bricxx::BadDelegateCall 11

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

bricxx::BadDelegateCall
Exception thrown by `Delegate::operator()()` if the delegate is not binded 11
bricxx::Delegate< TSignature >
A non-owning, const-noexcept-correct and equality comparable callable wrapper. Primary template, not defined 12
bricxx::Delegate< TRetVal(TParams...) const >
`Delegate` partial specialization for const, non-noexcept 14
bricxx::Delegate< TRetVal(TParams...) const noexcept >
`Delegate` partial specialization for const, noexcept 17
bricxx::Delegate< TRetVal(TParams...) noexcept >
`Delegate` partial specialization for non-const, noexcept 21
bricxx::Delegate< TRetVal(TParams...)>
`Delegate` partial specialization for non-const, non-noexcept 25
bricxx::DelegateBase< TRetVal, TParams >
Base class for `Delegate` partial specializations 29

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

delegate.hpp 33

Chapter 5

Namespace Documentation

5.1 bricxx Namespace Reference

Classes

- class Delegate

  A non-owning, const-noexcept-correct and equality comparable callable wrapper. Primary template, not defined.
- class BadDelegateCall

  Exception thrown by `Delegate::operator()()` if the delegate is not binded.
- class DelegateBase

  Base class for `Delegate` partial specializations.
- class Delegate< TRetVal(TParams...)>

  `Delegate` partial specialization for non-const, non-noexcept.
- class Delegate< TRetVal(TParams...) const >

  `Delegate` partial specialization for const, non-noexcept.
- class Delegate< TRetVal(TParams...) noexcept >

  `Delegate` partial specialization for non-const, noexcept.
- class Delegate< TRetVal(TParams...) const noexcept >

  `Delegate` partial specialization for const, noexcept.

Chapter 6

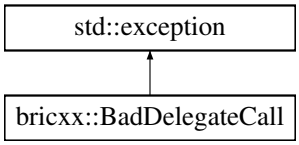
Class Documentation

6.1 bricxx::BadDelegateCall Class Reference

Exception thrown by `Delegate::operator()()` if the delegate is not binded.

`#include <delegate.hpp>`

Inheritance diagram for bricxx::BadDelegateCall:



Public Member Functions

- const char * what () const noexcept override

6.1.1 Detailed Description

Exception thrown by `Delegate::operator()()` if the delegate is not binded.

Definition at line 171 of file delegate.hpp.

6.1.2 Member Function Documentation

6.1.2.1 what()

```
const char * bricxx::BadDelegateCall::what () const  [override], [noexcept]
```

The documentation for this class was generated from the following file:

- delegate.hpp

6.2 bricxx::Delegate< TSignature > Class Template Reference

A non-owning, const-noexcept-correct and equality comparable callable wrapper. Primary template, not defined.

6.2.1 Detailed Description

template<typename TSignature>
class bricxx::Delegate< TSignature >

A non-owning, const-noexcept-correct and equality comparable callable wrapper. Primary template, not defined.

`Delegate`:

- is copyable, small (two pointers) and fast (no dynamic memory allocation, no virtual functions, all functions are inline (except stubs because their address is needed), delegate call involves only two extra calls (operator()() and stub function)).
- supports (non-member) functions, methods (member functions) and functors/lambda.
- supports compatible signatures.
- is a single header library so it is easy to integrate in any project.

Attention

Because `Delegate` is non-owning special care is needed to ensure that the binded target outlive it.

Remarks

- Two delegates are considered equal if they are not binded or binded to the same target.
- Equality comparison and `Delegate::IsBindedTo()` allows delegates to be easily used as callbacks that can be registered/unregistered (`::std::function_ref` is not equality comparable).
- The solutions presented in **references** directory are not const-noexcept-correct (they have the same problem as `::std::function`).

See also

- `Rich Hickey: Callbacks in C++ Using Template Functors`
- Bert Peers: Callbacks in C++
- Don Clugston, CodeProject: Member Function Pointers and the Fastest Possible C++ Delegates
- Sergey Ryazanov, CodeProject: The Impossibly Fast C++ Delegates
- Sergey Alexandrovich Kryukov, CodeProject: The Impossibly Fast C++ Delegates, Fixed
- Aleksei Trunov, CodeProject: Yet Another Generalized Functors Implementation in C++
- JaeWook Choi, CodeProject: Fast C++ Delegate
- `Matthew Rodusek: Creating a Fast and Efficient Delegate Type (Part 1)`
- `Matthew Rodusek: Creating a Fast and Efficient Delegate Type (Part 2)`
- `Matthew Rodusek: Creating a Fast and Efficient Delegate Type (Part 3)`

- Matthew Rodusek: Unbelievably Fast Delegate
- user1095108: Impossibly fast delegate in C++11
- user1095108: delegate.hpp
- **references** directory in this project that contains the above.

Minimal introduction/examples (please see tests for more details):

```
// (non-member) function
Delegate<int (int)> functionDelegate;
functionDelegate.Bind<&::islower>();
if (!functionDelegate.IsBindedTo<&::islower>()) throw runtime_error("function test failed");
if (!functionDelegate('a')) throw runtime_error("function test failed");

// lambda/functor
int numCalls = 0;
auto lambda = [&numCalls](int x, int y) mutable -> bool {
    ++numCalls;
    return x > y;
};
Delegate<bool (int, int)> lambdaDelegate;
lambdaDelegate.Bind(lambda);
if (!lambdaDelegate.IsBindedTo(lambda)) throw runtime_error("lambda test failed");
if (!lambdaDelegate(2, 1)) throw runtime_error("lambda test failed");
if (numCalls != 1) throw runtime_error("lambda test failed");

// method (member function)
string hello = "Hello, World!";
Delegate<void ()> methodDelegate;
methodDelegate.Bind<&string::clear>(hello);
methodDelegate();
if (!hello.empty()) throw runtime_error("method test failed");

// comparison
Delegate<int (int)> functionDelegate1;
functionDelegate1.Bind<&::isupper>();
if (functionDelegate == functionDelegate1) throw runtime_error("comparison test failed");
functionDelegate1.Bind<&::islower>();
if (functionDelegate != functionDelegate1) throw runtime_error("comparison test failed");

// const-correctness
Delegate<void () const> constDelegate;
//constDelegate.Bind<&string::clear>(hello);    // OK, does not compile
(void)constDelegate;

// noexcept-correctness
Delegate<void (char) noexcept> noexceptDelegate;
//noexceptDelegate.Bind<&string::push_back>(hello); // OK, does not compile
(void)noexceptDelegate;

// unbind
functionDelegate.Unbind();
if (functionDelegate.IsBinded()) throw runtime_error("unbind test failed");

// BadDelegateCall
bool caught = false;
try {
    (void)functionDelegate('a');
}
catch (const BadDelegateCall&) {
    caught = true;
}
if (!caught) throw runtime_error("BadDelegateCall test failed");

// compatible signatures
// long atol(const char* str);
// int atoi(const char* str);
Delegate<long (const char*)> delegate;
delegate.Bind<&::std::atoi>();
if (!delegate.IsBindedTo<&::std::atoi>()) throw runtime_error("compatible signatures test failed");
if (delegate("10") != 10) throw runtime_error("compatible signatures test failed");
```

Definition at line 164 of file delegate.hpp.

The documentation for this class was generated from the following file:

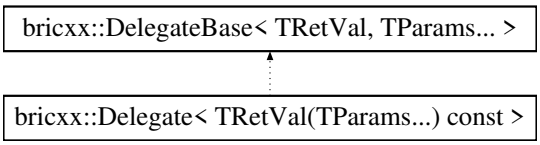
- delegate.hpp

6.3 bricxx::Delegate< TRetVal(TParams...) const > Class Template Reference

`Delegate` partial specialization for const, non-noexcept.

`#include <delegate.hpp>`

Inheritance diagram for bricxx::Delegate< TRetVal(TParams...) const >:



Public Member Functions

- ~Delegate () noexcept
- constexpr Delegate & operator= (const Delegate &other) noexcept
- template<typename... UParams, typename = ::std::enable_if_t< ::std::is_invocable_v<TRetVal (TParams...), UParams...> >> TRetVal operator() (UParams &&... params) const

Constructors.

- constexpr Delegate () noexcept
  Creates a delegate that is not binded.
- constexpr Delegate (const Delegate &other) noexcept

Equality operators.

- constexpr bool operator== (const Delegate &other) const noexcept
- constexpr bool operator!= (const Delegate &other) const noexcept

Binders.

Binds the delegate to a target.

- template<class TFunctor, typename = ::std::enable_if_t< ::std::is_class_v<TFunctor> && ::std::is_invocable_r_v<TRetVal, const TFunctor&, TParams...> >> constexpr void Bind (const TFunctor &functor) noexcept
  Binds the delegate to a `functor` / lambda (`TFunctor` is auto-deduced).
- template<auto TMethod, class TClass, typename = ::std::enable_if_t< ::std::is_invocable_r_v<TRetVal, decltype(TMethod), const TClass&, TParams...> >> constexpr void Bind (const TClass &object) noexcept

Binder target testers (convenience methods).

Test if the delegate is binded to the given target.

- template<class TFunctor, typename = ::std::enable_if_t< ::std::is_class_v<TFunctor> && ::std::is_invocable_r_v<TRetVal, const TFunctor&, TParams...> >> constexpr bool IsBindedTo (const TFunctor &functor) const noexcept
  Test if the delegate is binded to the given `functor` / lambda (`TFunctor` is auto-deduced).
- template<auto TMethod, class TClass, typename = ::std::enable_if_t< ::std::is_invocable_r_v<TRetVal, decltype(TMethod), const TClass&, TParams...> >> constexpr bool IsBindedTo (const TClass &object) const noexcept

6.3.1 Detailed Description

template<typename TRetVal, typename... TParams>
class bricxx::Delegate< TRetVal(TParams...) const >

`Delegate` partial specialization for const, non-noexcept.

Definition at line 343 of file delegate.hpp.

6.3.2 Constructor & Destructor Documentation

6.3.2.1 Delegate() [1/2]

```
template<typename TRetVal, typename...  TParams>
bricxx::Delegate< TRetVal(TParams...)  const >::Delegate ()  [constexpr], [noexcept]
```

Creates a delegate that is not binded.

6.3.2.2 Delegate() [2/2]

```
template<typename TRetVal, typename...  TParams>
bricxx::Delegate< TRetVal(TParams...)  const >::Delegate (
            const Delegate< TRetVal(TParams...)  const > & other)  [constexpr], [noexcept]
```

6.3.2.3 ~Delegate()

```
template<typename TRetVal, typename...  TParams>
bricxx::Delegate< TRetVal(TParams...)  const >::~Delegate ()  [noexcept]
```

Remarks

Not virtual.

6.3.3 Member Function Documentation

6.3.3.1 operator=()

```
template<typename TRetVal, typename...  TParams>
Delegate & bricxx::Delegate< TRetVal(TParams...)  const >::operator= (
            const Delegate< TRetVal(TParams...)  const > & other)  [constexpr], [noexcept]
```

6.3.3.2 operator==()

```
template<typename TRetVal, typename...  TParams>
bool bricxx::Delegate< TRetVal(TParams...)  const >::operator== (
            const Delegate< TRetVal(TParams...)  const > & other) const  [nodiscard], [constexpr],
[noexcept]
```

6.3.3.3 operator"!=()

```
template<typename TRetVal, typename... TParams>
bool bricxx::Delegate< TRetVal(TParams...) const >::operator!= (
            const Delegate< TRetVal(TParams...) const > & other) const [nodiscard], [constexpr],
[noexcept]
```

6.3.3.4 Bind() [1/2]

```
template<typename TRetVal, typename... TParams>
template<class TFunctor, typename = ::std::enable_if_t< ::std::is_class_v<TFunctor> && ::↩
std::is_invocable_r_v<TRetVal, const TFunctor&, TParams...> >>
void bricxx::Delegate< TRetVal(TParams...) const >::Bind (
            const TFunctor & functor) [constexpr], [noexcept]
```

Binds the delegate to a `functor` / lambda (`TFunctor` is auto-deduced).

6.3.3.5 Bind() [2/2]

```
template<typename TRetVal, typename... TParams>
template<auto TMethod, class TClass, typename = ::std::enable_if_t< ::std::is_invocable_r_↩
v<TRetVal, decltype(TMethod), const TClass&, TParams...> >>
void bricxx::Delegate< TRetVal(TParams...) const >::Bind (
            const TClass & object) [constexpr], [noexcept]
```

Binds the delegate to an `object` + method (member function).

Remarks

Uses C++17 auto for template parameters to support compatible signatures and auto-deduce `TClass`

6.3.3.6 IsBindedTo() [1/2]

```
template<typename TRetVal, typename... TParams>
template<class TFunctor, typename = ::std::enable_if_t< ::std::is_class_v<TFunctor> && ::↩
std::is_invocable_r_v<TRetVal, const TFunctor&, TParams...> >>
bool bricxx::Delegate< TRetVal(TParams...) const >::IsBindedTo (
            const TFunctor & functor) const [nodiscard], [constexpr], [noexcept]
```

Test if the delegate is binded to the given `functor` / lambda (`TFunctor` is auto-deduced).

6.3.3.7 IsBindedTo() [2/2]

```
template<typename TRetVal, typename... TParams>
template<auto TMethod, class TClass, typename = ::std::enable_if_t< ::std::is_invocable_r_↩
v<TRetVal, decltype(TMethod), const TClass&, TParams...> >>
bool bricxx::Delegate< TRetVal(TParams...) const >::IsBindedTo (
            const TClass & object) const [nodiscard], [constexpr], [noexcept]
```

Test if the delegate is binded to the given `object` + method (member function).

Remarks

Uses C++17 auto for template parameters to support compatible signatures and auto-deduce `TClass`

6.3.3.8 operator()()

```
template<typename TRetVal, typename... TParams>
template<typename... UParams, typename = ::std::enable_if_t< ::std::is_invocable_v<TRetVal
(TParams...), UParams...> >>
TRetVal bricxx::Delegate< TRetVal(TParams...) const >::operator() (
            UParams &&... params) const [nodiscard]
```

Invokes the stub function with the given parameters.

Remarks

Perfect forwarding and converting parameter pack to stub function (`UParams` is a deduced type, `TParams` is not).

Exceptions

BadDelegateCall	if the delegate is not binded.

The documentation for this class was generated from the following file:

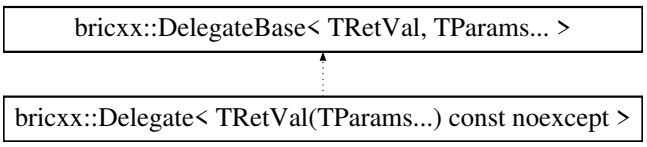
- delegate.hpp

6.4 bricxx::Delegate< TRetVal(TParams...) const noexcept > Class Template Reference

`Delegate` partial specialization for const, noexcept.

```
#include <delegate.hpp>
```

Inheritance diagram for bricxx::Delegate< TRetVal(TParams...) const noexcept >:



Public Member Functions

- ~Delegate () noexcept
- constexpr Delegate & operator= (const Delegate &other) noexcept
- template<typename... UParams, typename = ::std::enable_if_t< ::std::is_nothrow_invocable_v<TRetVal (TParams...) noexcept, UParams...> >>
  TRetVal operator() (UParams &&... params) const noexcept

Constructors.

- constexpr Delegate () noexcept

Creates a delegate that is not binded.

- constexpr Delegate (const Delegate &other) noexcept

Equality operators.

- constexpr bool operator== (const Delegate &other) const noexcept
- constexpr bool operator!= (const Delegate &other) const noexcept

Binders.

Binds the delegate to a target.

- template<class TFunctor, typename = ::std::enable_if_t< ::std::is_class_v<TFunctor> && ::std::is_nothrow_invocable_r_v<↩ TRetVal, const TFunctor&, TParams...> >>
  constexpr void Bind (const TFunctor &functor) noexcept
  Binds the delegate to a `functor` / lambda (`TFunctor` is auto-deduced).
- template<auto TMethod, class TClass, typename = ::std::enable_if_t< ::std::is_nothrow_invocable_r_v<TRetVal, decltype(↩ TMethod), const TClass&, TParams...> >>
  constexpr void Bind (const TClass &object) noexcept

Binder target testers (convenience methods).

Test if the delegate is binded to the given target.

- template<class TFunctor, typename = ::std::enable_if_t< ::std::is_class_v<TFunctor> && ::std::is_nothrow_invocable_r_v<↩ TRetVal, const TFunctor&, TParams...> >>
  constexpr bool IsBindedTo (const TFunctor &functor) const noexcept
  Test if the delegate is binded to the given `functor` / lambda (`TFunctor` is auto-deduced).
- template<auto TMethod, class TClass, typename = ::std::enable_if_t< ::std::is_nothrow_invocable_r_v<TRetVal, decltype(↩ TMethod), const TClass&, TParams...> >>
  constexpr bool IsBindedTo (const TClass &object) const noexcept

6.4.1 Detailed Description

template<typename TRetVal, typename... TParams>
class bricxx::Delegate< TRetVal(TParams...) const noexcept >

`Delegate` partial specialization for const, noexcept.

Definition at line 557 of file delegate.hpp.

6.4.2 Constructor & Destructor Documentation

6.4.2.1 Delegate() [1/2]

```
template<typename TRetVal, typename... TParams>
bricxx::Delegate< TRetVal(TParams...) const noexcept >::Delegate () [constexpr], [noexcept]
```

Creates a delegate that is not binded.

6.4.2.2 Delegate() [2/2]

```
template<typename TRetVal, typename... TParams>
bricxx::Delegate< TRetVal(TParams...) const noexcept >::Delegate (
            const Delegate< TRetVal(TParams...) const noexcept > & other) [constexpr],
[noexcept]
```

6.4.2.3 ~Delegate()

```
template<typename TRetVal, typename... TParams>
bricxx::Delegate< TRetVal(TParams...) const noexcept >::~Delegate () [noexcept]
```

Remarks

Not virtual.

6.4.3 Member Function Documentation

6.4.3.1 operator=()

```
template<typename TRetVal, typename... TParams>
Delegate & bricxx::Delegate< TRetVal(TParams...) const noexcept >::operator= (
            const Delegate< TRetVal(TParams...) const noexcept > & other) [constexpr],
[noexcept]
```

6.4.3.2 operator==()

```
template<typename TRetVal, typename... TParams>
bool bricxx::Delegate< TRetVal(TParams...) const noexcept >::operator== (
            const Delegate< TRetVal(TParams...) const noexcept > & other) const [nodiscard],
[constexpr], [noexcept]
```

6.4.3.3 operator"!=()

```
template<typename TRetVal, typename... TParams>
bool bricxx::Delegate< TRetVal(TParams...) const noexcept >::operator!= (
            const Delegate< TRetVal(TParams...) const noexcept > & other) const [nodiscard],
[constexpr], [noexcept]
```

6.4.3.4 Bind() [1/2]

```
template<typename TRetVal, typename... TParams>
template<class TFunctor, typename = ::std::enable_if_t< ::std::is_class_v<TFunctor> && ::↩
std::is_nothrow_invocable_r_v<TRetVal, const TFunctor&, TParams...> >>
void bricxx::Delegate< TRetVal(TParams...) const noexcept >::Bind (
            const TFunctor & functor) [constexpr], [noexcept]
```

Binds the delegate to a `functor` / lambda (`TFunctor` is auto-deduced).

6.4.3.5 Bind() [2/2]

```
template<typename TRetVal, typename...  TParams>
template<auto TMethod, class TClass, typename = ::std::enable_if_t< ::std::is_nothrow_invocable↩
_r_v<TRetVal, decltype(TMethod), const TClass&, TParams...> >>
void bricxx::Delegate< TRetVal(TParams...)  const noexcept >::Bind (
            const TClass & object)  [constexpr], [noexcept]
```

Binds the delegate to an `object` + method (member function).

Remarks

Uses C++17 auto for template parameters to support compatible signatures and auto-deduce `TClass`

6.4.3.6 IsBindedTo() [1/2]

```
template<typename TRetVal, typename...  TParams>
template<class TFunctor, typename = ::std::enable_if_t< ::std::is_class_v<TFunctor> && ::↩
std::is_nothrow_invocable_r_v<TRetVal, const TFunctor&, TParams...> >>
bool bricxx::Delegate< TRetVal(TParams...)  const noexcept >::IsBindedTo (
            const TFunctor & functor) const  [nodiscard], [constexpr], [noexcept]
```

Test if the delegate is binded to the given `functor` / lambda (`TFunctor` is auto-deduced).

6.4.3.7 IsBindedTo() [2/2]

```
template<typename TRetVal, typename...  TParams>
template<auto TMethod, class TClass, typename = ::std::enable_if_t< ::std::is_nothrow_invocable↩
_r_v<TRetVal, decltype(TMethod), const TClass&, TParams...> >>
bool bricxx::Delegate< TRetVal(TParams...)  const noexcept >::IsBindedTo (
            const TClass & object) const  [nodiscard], [constexpr], [noexcept]
```

Test if the delegate is binded to the given `object` + method (member function).

Remarks

Uses C++17 auto for template parameters to support compatible signatures and auto-deduce `TClass`

6.4.3.8 operator()()

```
template<typename TRetVal, typename...  TParams>
template<typename...  UParams, typename = ::std::enable_if_t< ::std::is_nothrow_invocable_↩
v<TRetVal (TParams...)  noexcept, UParams...> >>
TRetVal bricxx::Delegate< TRetVal(TParams...)  const noexcept >::operator() (
            UParams &&...  params) const  [nodiscard], [noexcept]
```

Invokes the stub function with the given parameters.

Remarks

Perfect forwarding and converting parameter pack to stub function (`UParams` is a deduced type, `TParams` is not).

Attention

Calls `::std::terminate()` if the delegate is not binded.

The documentation for this class was generated from the following file:

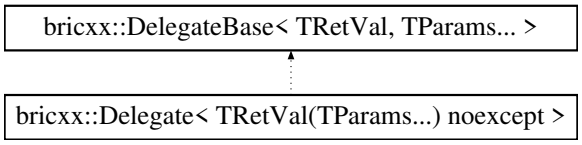
- delegate.hpp

6.5 bricxx::Delegate< TRetVal(TParams...) noexcept > Class Template Reference

`Delegate` partial specialization for non-const, noexcept.

`#include <delegate.hpp>`

Inheritance diagram for bricxx::Delegate< TRetVal(TParams...) noexcept >:



Public Member Functions

- ~Delegate () noexcept
- constexpr Delegate & operator= (const Delegate &other) noexcept
- template<typename... UParams, typename = ::std::enable_if_t< ::std::is_nothrow_invocable_v<TRetVal (TParams...) noexcept, UParams...> >>
  TRetVal operator() (UParams &&... params) noexcept

Constructors.

- constexpr Delegate () noexcept
  Creates a delegate that is not binded.
- constexpr Delegate (const Delegate &other) noexcept

Equality operators.

- constexpr bool operator== (const Delegate &other) const noexcept
- constexpr bool operator!= (const Delegate &other) const noexcept

Binders.

Binds the delegate to a target.

- template<auto TFunction, typename = ::std::enable_if_t< ::std::is_nothrow_invocable_r_v<TRetVal, decltype(TFunction), TParams...> >>
  constexpr void Bind () noexcept
- template<class TFunctor, typename = ::std::enable_if_t< ::std::is_class_v<TFunctor> && ::std::is_nothrow_invocable_r_v<↩ TRetVal, TFunctor&, TParams...> >>
  constexpr void Bind (TFunctor &functor) noexcept
  Binds the delegate to a `functor` / lambda (`TFunctor` is auto-deduced).
- template<auto TMethod, class TClass, typename = ::std::enable_if_t< ::std::is_nothrow_invocable_r_v<TRetVal, decltype(↩ TMethod), TClass&, TParams...> >>
  constexpr void Bind (TClass &object) noexcept

Binder target testers (convenience methods).

Test if the delegate is binded to the given target.

- template<auto TFunction, typename = ::std::enable_if_t< ::std::is_nothrow_invocable_r_v<TRetVal, decltype(TFunction), TParams...> >>
  constexpr bool IsBindedTo () const noexcept
- template<class TFunctor, typename = ::std::enable_if_t< ::std::is_class_v<TFunctor> && ::std::is_nothrow_invocable_r_v<↩ TRetVal, TFunctor&, TParams...> >>
  constexpr bool IsBindedTo (TFunctor &functor) const noexcept
  Test if the delegate is binded to the given `functor` / lambda (`TFunctor` is auto-deduced).
- template<auto TMethod, class TClass, typename = ::std::enable_if_t< ::std::is_nothrow_invocable_r_v<TRetVal, decltype(↩ TMethod), TClass&, TParams...> >>
  constexpr bool IsBindedTo (TClass &object) const noexcept

6.5.1 Detailed Description

template<typename TRetVal, typename... TParams>
class bricxx::Delegate< TRetVal(TParams...) noexcept >

`Delegate` partial specialization for non-const, noexcept.

Definition at line 441 of file delegate.hpp.

6.5.2 Constructor & Destructor Documentation

6.5.2.1 Delegate() [1/2]

```
template<typename TRetVal, typename...  TParams>
bricxx::Delegate< TRetVal(TParams...)  noexcept >::Delegate ()  [constexpr], [noexcept]
```

Creates a delegate that is not binded.

6.5.2.2 Delegate() [2/2]

```
template<typename TRetVal, typename...  TParams>
bricxx::Delegate< TRetVal(TParams...)  noexcept >::Delegate (
            const Delegate< TRetVal(TParams...)  noexcept > & other)  [constexpr], [noexcept]
```

6.5.2.3 ~Delegate()

```
template<typename TRetVal, typename...  TParams>
bricxx::Delegate< TRetVal(TParams...)  noexcept >::~Delegate ()  [noexcept]
```

Remarks

Not virtual.

6.5.3 Member Function Documentation

6.5.3.1 operator=()

```
template<typename TRetVal, typename...  TParams>
Delegate & bricxx::Delegate< TRetVal(TParams...)  noexcept >::operator= (
            const Delegate< TRetVal(TParams...)  noexcept > & other)  [constexpr], [noexcept]
```

6.5.3.2 operator==()

```
template<typename TRetVal, typename...  TParams>
bool bricxx::Delegate< TRetVal(TParams...)  noexcept >::operator== (
            const Delegate< TRetVal(TParams...)  noexcept > & other) const  [nodiscard],
[constexpr], [noexcept]
```

6.5.3.3 operator"!=()

```
template<typename TRetVal, typename...  TParams>
bool bricxx::Delegate< TRetVal(TParams...)  noexcept >::operator!= (
            const Delegate< TRetVal(TParams...)  noexcept > & other) const  [nodiscard],
[constexpr], [noexcept]
```

6.5.3.4 Bind() [1/3]

```
template<typename TRetVal, typename...  TParams>
template<auto TFunction, typename = ::std::enable_if_t< ::std::is_nothrow_invocable_r_v<↩
TRetVal, decltype(TFunction), TParams...> >>
void bricxx::Delegate< TRetVal(TParams...)  noexcept >::Bind ()  [constexpr], [noexcept]
```

Binds the delegate to a (non-member) function.

Remarks

Uses C++17 auto for template parameters to support compatible signatures.

6.5.3.5 Bind() [2/3]

```
template<typename TRetVal, typename...  TParams>
template<class TFunctor, typename = ::std::enable_if_t< ::std::is_class_v<TFunctor> && ::↩
std::is_nothrow_invocable_r_v<TRetVal, TFunctor&, TParams...> >>
void bricxx::Delegate< TRetVal(TParams...)  noexcept >::Bind (
            TFunctor & functor)  [constexpr], [noexcept]
```

Binds the delegate to a `functor` / lambda (`TFunctor` is auto-deduced).

6.5.3.6 Bind() [3/3]

```
template<typename TRetVal, typename...  TParams>
template<auto TMethod, class TClass, typename = ::std::enable_if_t< ::std::is_nothrow_invocable↩
_r_v<TRetVal, decltype(TMethod), TClass&, TParams...> >>
void bricxx::Delegate< TRetVal(TParams...)  noexcept >::Bind (
            TClass & object)  [constexpr], [noexcept]
```

Binds the delegate to an `object` + method (member function).

Remarks

Uses C++17 auto for template parameters to support compatible signatures and auto-deduce `TClass`

6.5.3.7 IsBindedTo() [1/3]

```
template<typename TRetVal, typename... TParams>
template<auto TFunction, typename = ::std::enable_if_t< ::std::is_nothrow_invocable_r_v<↩
TRetVal, decltype(TFunction), TParams...> >>
bool bricxx::Delegate< TRetVal(TParams...) noexcept >::IsBindedTo () const [nodiscard],
[constexpr], [noexcept]
```

Test if the delegate is binded to the given (non-member) function.

Remarks

Uses C++17 auto for template parameters to support compatible signatures.

6.5.3.8 IsBindedTo() [2/3]

```
template<typename TRetVal, typename... TParams>
template<class TFunctor, typename = ::std::enable_if_t< ::std::is_class_v<TFunctor> && ::↩
std::is_nothrow_invocable_r_v<TRetVal, TFunctor&, TParams...> >>
bool bricxx::Delegate< TRetVal(TParams...) noexcept >::IsBindedTo (
            TFunctor & functor) const [nodiscard], [constexpr], [noexcept]
```

Test if the delegate is binded to the given `functor` / lambda (`TFunctor` is auto-deduced).

6.5.3.9 IsBindedTo() [3/3]

```
template<typename TRetVal, typename... TParams>
template<auto TMethod, class TClass, typename = ::std::enable_if_t< ::std::is_nothrow_invocable↩
_r_v<TRetVal, decltype(TMethod), TClass&, TParams...> >>
bool bricxx::Delegate< TRetVal(TParams...) noexcept >::IsBindedTo (
            TClass & object) const [nodiscard], [constexpr], [noexcept]
```

Test if the delegate is binded to the given `object` + method (member function).

Remarks

Uses C++17 auto for template parameters to support compatible signatures and auto-deduce `TClass`

6.5.3.10 operator()()

```
template<typename TRetVal, typename... TParams>
template<typename... UParams, typename = ::std::enable_if_t< ::std::is_nothrow_invocable_↩
v<TRetVal (TParams...) noexcept, UParams...> >>
TRetVal bricxx::Delegate< TRetVal(TParams...) noexcept >::operator() (
            UParams &&... params) [nodiscard], [noexcept]
```

Invokes the stub function with the given parameters.

Remarks

Perfect forwarding and converting parameter pack to stub function (`UParams` is a deduced type, `TParams` is not).

Attention

Calls `::std::terminate()` if the delegate is not binded.

The documentation for this class was generated from the following file:

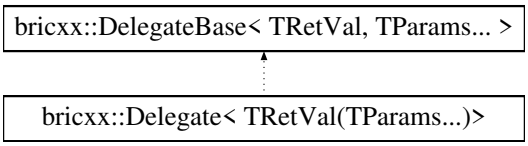
- delegate.hpp

6.6 bricxx::Delegate< TRetVal(TParams...)> Class Template Reference

`Delegate` partial specialization for non-const, non-noexcept.

`#include <delegate.hpp>`

Inheritance diagram for bricxx::Delegate< TRetVal(TParams...)>:



Public Member Functions

- ~Delegate () noexcept
- constexpr Delegate & operator= (const Delegate &other) noexcept
- template<typename... UParams, typename = ::std::enable_if_t< ::std::is_invocable_v<TRetVal (TParams...), UParams...> >> TRetVal operator() (UParams &&... params)

Constructors.

- constexpr Delegate () noexcept
  Creates a delegate that is not binded.
- constexpr Delegate (const Delegate &other) noexcept

Equality operators.

- constexpr bool operator== (const Delegate &other) const noexcept
- constexpr bool operator!= (const Delegate &other) const noexcept

Binders.

Binds the delegate to a target.

- template<auto TFunction, typename = ::std::enable_if_t< ::std::is_invocable_r_v<TRetVal, decltype(TFunction), TParams...> >> constexpr void Bind () noexcept
- template<class TFunctor, typename = ::std::enable_if_t< ::std::is_class_v<TFunctor> && ::std::is_invocable_r_v<TRetVal, TFunctor&, TParams...> >> constexpr void Bind (TFunctor &functor) noexcept
  Binds the delegate to a `functor` / lambda (`TFunctor` is auto-deduced).
- template<auto TMethod, class TClass, typename = ::std::enable_if_t< ::std::is_invocable_r_v<TRetVal, decltype(TMethod), TClass&, TParams...> >> constexpr void Bind (TClass &object) noexcept

Binder target testers (convenience methods).

Test if the delegate is binded to the given target.

- template<auto TFunction, typename = ::std::enable_if_t< ::std::is_invocable_r_v<TRetVal, decltype(TFunction), TParams...> >> constexpr bool IsBindedTo () const noexcept
- template<class TFunctor, typename = ::std::enable_if_t< ::std::is_class_v<TFunctor> && ::std::is_invocable_r_v<TRetVal, TFunctor&, TParams...> >> constexpr bool IsBindedTo (TFunctor &functor) const noexcept
  Test if the delegate is binded to the given `functor` / lambda (`TFunctor` is auto-deduced).
- template<auto TMethod, class TClass, typename = ::std::enable_if_t< ::std::is_invocable_r_v<TRetVal, decltype(TMethod), TClass&, TParams...> >> constexpr bool IsBindedTo (TClass &object) const noexcept

6.6.1 Detailed Description

template<typename TRetVal, typename... TParams>
class bricxx::Delegate< TRetVal(TParams...)>

`Delegate` partial specialization for non-const, non-noexcept.

Definition at line 227 of file delegate.hpp.

6.6.2 Constructor & Destructor Documentation

6.6.2.1 Delegate() [1/2]

```
template<typename TRetVal, typename...  TParams>
bricxx::Delegate< TRetVal(TParams...)>::Delegate ()  [constexpr], [noexcept]
```

Creates a delegate that is not binded.

6.6.2.2 Delegate() [2/2]

```
template<typename TRetVal, typename...  TParams>
bricxx::Delegate< TRetVal(TParams...)>::Delegate (
            const Delegate< TRetVal(TParams...)> & other)  [constexpr], [noexcept]
```

6.6.2.3 ~Delegate()

```
template<typename TRetVal, typename...  TParams>
bricxx::Delegate< TRetVal(TParams...)>::~Delegate ()  [noexcept]
```

Remarks

Not virtual.

6.6.3 Member Function Documentation

6.6.3.1 operator=()

```
template<typename TRetVal, typename...  TParams>
Delegate & bricxx::Delegate< TRetVal(TParams...)>::operator= (
            const Delegate< TRetVal(TParams...)> & other)  [constexpr], [noexcept]
```

6.6.3.2 operator==()

```
template<typename TRetVal, typename...  TParams>
bool bricxx::Delegate< TRetVal(TParams...)>::operator== (
            const Delegate< TRetVal(TParams...)> & other) const  [nodiscard], [constexpr],
[noexcept]
```

6.6.3.3 operator"!=()

```
template<typename TRetVal, typename...  TParams>
bool bricxx::Delegate< TRetVal(TParams...)>::operator!= (
            const Delegate< TRetVal(TParams...)> & other) const  [nodiscard], [constexpr],
[noexcept]
```

6.6.3.4 Bind() [1/3]

```
template<typename TRetVal, typename...  TParams>
template<auto TFunction, typename = ::std::enable_if_t< ::std::is_invocable_r_v<TRetVal,
decltype(TFunction), TParams...> >>
void bricxx::Delegate< TRetVal(TParams...)>::Bind ()  [constexpr], [noexcept]
```

Binds the delegate to a (non-member) function.

Remarks

Uses C++17 auto for template parameters to support compatible signatures.

6.6.3.5 Bind() [2/3]

```
template<typename TRetVal, typename...  TParams>
template<class TFunctor, typename = ::std::enable_if_t< ::std::is_class_v<TFunctor> && ::↩
std::is_invocable_r_v<TRetVal, TFunctor&, TParams...> >>
void bricxx::Delegate< TRetVal(TParams...)>::Bind (
            TFunctor & functor)  [constexpr], [noexcept]
```

Binds the delegate to a `functor` / lambda (`TFunctor` is auto-deduced).

6.6.3.6 Bind() [3/3]

```
template<typename TRetVal, typename...  TParams>
template<auto TMethod, class TClass, typename = ::std::enable_if_t< ::std::is_invocable_r_↩
v<TRetVal, decltype(TMethod), TClass&, TParams...> >>
void bricxx::Delegate< TRetVal(TParams...)>::Bind (
            TClass & object)  [constexpr], [noexcept]
```

Binds the delegate to an `object` + method (member function).

Remarks

Uses C++17 auto for template parameters to support compatible signatures and auto-deduce `TClass`

6.6.3.7 IsBindedTo() [1/3]

```
template<typename TRetVal, typename...  TParams>
template<auto TFunction, typename = ::std::enable_if_t< ::std::is_invocable_r_v<TRetVal,
decltype(TFunction), TParams...> >>
bool bricxx::Delegate< TRetVal(TParams...)>::IsBindedTo () const  [nodiscard], [constexpr],
[noexcept]
```

Test if the delegate is binded to the given (non-member) function.

Remarks

Uses C++17 auto for template parameters to support compatible signatures.

6.6.3.8 IsBindedTo() [2/3]

```
template<typename TRetVal, typename...  TParams>
template<class TFunctor, typename = ::std::enable_if_t< ::std::is_class_v<TFunctor> && ::↩
std::is_invocable_r_v<TRetVal, TFunctor&, TParams...> >>
bool bricxx::Delegate< TRetVal(TParams...)>::IsBindedTo (
            TFunctor & functor) const  [nodiscard], [constexpr], [noexcept]
```

Test if the delegate is binded to the given `functor` / lambda (`TFunctor` is auto-deduced).

6.6.3.9 IsBindedTo() [3/3]

```
template<typename TRetVal, typename...  TParams>
template<auto TMethod, class TClass, typename = ::std::enable_if_t< ::std::is_invocable_r_↩
v<TRetVal, decltype(TMethod), TClass&, TParams...> >>
bool bricxx::Delegate< TRetVal(TParams...)>::IsBindedTo (
            TClass & object) const  [nodiscard], [constexpr], [noexcept]
```

Test if the delegate is binded to the given `object` + method (member function).

Remarks

Uses C++17 auto for template parameters to support compatible signatures and auto-deduce `TClass`

6.6.3.10 operator()()

```
template<typename TRetVal, typename...  TParams>
template<typename...  UParams, typename = ::std::enable_if_t< ::std::is_invocable_v<TRetVal
(TParams...), UParams...> >>
TRetVal bricxx::Delegate< TRetVal(TParams...)>::operator() (
            UParams &&...  params)  [nodiscard]
```

Invokes the stub function with the given parameters.

Remarks

Perfect forwarding and converting parameter pack to stub function (`UParams` is a deduced type, `TParams` is not).

Exceptions

BadDelegateCall	if the delegate is not binded.

The documentation for this class was generated from the following file:

- delegate.hpp

6.7 bricxx::DelegateBase< TRetVal, TParams > Class Template Reference

Base class for `Delegate` partial specializations.

`#include <delegate.hpp>`

Public Member Functions

- constexpr bool IsBinded () const noexcept
- constexpr operator bool () const noexcept

  Same as `IsBinded()`.
- constexpr void Unbind () noexcept
- constexpr void Reset () noexcept

  Same as `Unbind()`.

Protected Types

- using ErasedObjectType = void
- using StubFnPtr = TRetVal (∗)(ErasedObjectType∗ pTypeErasedObject, TParams... params)

Protected Member Functions

- ∼DelegateBase () noexcept
- constexpr DelegateBase & operator= (const DelegateBase &other) noexcept

Constructors.

- constexpr DelegateBase () noexcept

  Creates a delegate that is not binded.
- constexpr DelegateBase (const DelegateBase &other) noexcept

Equality operators.

- constexpr bool operator== (const DelegateBase &other) const noexcept
- constexpr bool operator!= (const DelegateBase &other) const noexcept

Static Protected Member Functions

- static TRetVal NullStub (ErasedObjectType ∗pTypeErasedObject, TParams... params)

Protected Attributes

- StubFnPtr pStub_ = &NullStub
- ErasedObjectType ∗ pTypeErasedObject_ = nullptr

6.7.1 Detailed Description

template<typename TRetVal, typename... TParams>
class bricxx::DelegateBase< TRetVal, TParams >

Base class for `Delegate` partial specializations.

Definition at line 182 of file delegate.hpp.

6.7.2 Member Typedef Documentation

6.7.2.1 ErasedObjectType

```
template<typename TRetVal, typename...  TParams>
using bricxx::DelegateBase< TRetVal, TParams >::ErasedObjectType = void  [protected]
```

Definition at line 190 of file delegate.hpp.

6.7.2.2 StubFnPtr

```
template<typename TRetVal, typename...  TParams>
using bricxx::DelegateBase< TRetVal, TParams >::StubFnPtr = TRetVal (*)(ErasedObjectType* p↩
TypeErasedObject, TParams...  params)  [protected]
```

Definition at line 191 of file delegate.hpp.

6.7.3 Constructor & Destructor Documentation

6.7.3.1 DelegateBase() [1/2]

```
template<typename TRetVal, typename...  TParams>
bricxx::DelegateBase< TRetVal, TParams >::DelegateBase ()  [constexpr], [protected], [noexcept]
```

Creates a delegate that is not binded.

6.7.3.2 DelegateBase() [2/2]

```
template<typename TRetVal, typename...  TParams>
bricxx::DelegateBase< TRetVal, TParams >::DelegateBase (
            const DelegateBase< TRetVal, TParams > & other)  [constexpr], [protected], [noexcept]
```

6.7.3.3 ~DelegateBase()

```
template<typename TRetVal, typename...  TParams>
bricxx::DelegateBase< TRetVal, TParams >::~DelegateBase ()  [protected], [noexcept]
```

Remarks

Not virtual.

6.7.4 Member Function Documentation

6.7.4.1 IsBinded()

```
template<typename TRetVal, typename...  TParams>
bool bricxx::DelegateBase< TRetVal, TParams >::IsBinded () const  [nodiscard], [constexpr],
[noexcept]
```

6.7.4.2 operator bool()

```
template<typename TRetVal, typename...  TParams>
bricxx::DelegateBase< TRetVal, TParams >::operator bool () const  [explicit], [nodiscard],
[constexpr], [noexcept]
```

Same as `IsBinded()`.

6.7.4.3 Unbind()

```
template<typename TRetVal, typename...  TParams>
void bricxx::DelegateBase< TRetVal, TParams >::Unbind ()  [constexpr], [noexcept]
```

6.7.4.4 Reset()

```
template<typename TRetVal, typename...  TParams>
void bricxx::DelegateBase< TRetVal, TParams >::Reset ()  [constexpr], [noexcept]
```

Same as `Unbind()`.

6.7.4.5 operator=()

```
template<typename TRetVal, typename...  TParams>
DelegateBase & bricxx::DelegateBase< TRetVal, TParams >::operator= (
            const DelegateBase< TRetVal, TParams > & other)  [constexpr], [protected], [noexcept]
```

6.7.4.6 operator==()

```
template<typename TRetVal, typename...  TParams>
bool bricxx::DelegateBase< TRetVal, TParams >::operator== (
            const DelegateBase< TRetVal, TParams > & other) const  [nodiscard], [constexpr],
[protected], [noexcept]
```

6.7.4.7 operator"!=()

```
template<typename TRetVal, typename...  TParams>
bool bricxx::DelegateBase< TRetVal, TParams >::operator!= (
            const DelegateBase< TRetVal, TParams > & other) const  [nodiscard], [constexpr],
[protected], [noexcept]
```

6.7.4.8 NullStub()

```
template<typename TRetVal, typename... TParams>
TRetVal bricxx::DelegateBase< TRetVal, TParams >::NullStub (
            ErasedObjectType * pTypeErasedObject,
            TParams... params) [static], [protected]
```

Exceptions

BadDelegateCall	This stub is used when the delegate is not binded to a "real" target.

6.7.5 Member Data Documentation

6.7.5.1 pStub_

```
template<typename TRetVal, typename... TParams>
StubFnPtr bricxx::DelegateBase< TRetVal, TParams >::pStub_ = &NullStub [protected]
```

Definition at line 217 of file delegate.hpp.

6.7.5.2 pTypeErasedObject_

```
template<typename TRetVal, typename... TParams>
ErasedObjectType* bricxx::DelegateBase< TRetVal, TParams >::pTypeErasedObject_ = nullptr [protected]
```

Definition at line 218 of file delegate.hpp.

The documentation for this class was generated from the following file:

- delegate.hpp

Chapter 7

File Documentation

7.1 delegate.hpp File Reference

```
#include <exception>
#include <type_traits>
#include <utility>
#include <memory>
```

Classes

- class bricxx::BadDelegateCall

  Exception thrown by `Delegate::operator()()` if the delegate is not binded.
- class bricxx::DelegateBase< TRetVal, TParams >

  Base class for `Delegate` partial specializations.
- class bricxx::Delegate< TRetVal(TParams...)>

  `Delegate` partial specialization for non-const, non-noexcept.
- class bricxx::Delegate< TRetVal(TParams...) const >

  `Delegate` partial specialization for const, non-noexcept.
- class bricxx::Delegate< TRetVal(TParams...) noexcept >

  `Delegate` partial specialization for non-const, noexcept.
- class bricxx::Delegate< TRetVal(TParams...) const noexcept >

  `Delegate` partial specialization for const, noexcept.

Namespaces

- namespace bricxx

Macros

- #define BRICXX_DELEGATE_API

7.1.1 Macro Definition Documentation

7.1.1.1 BRICXX_DELEGATE_API

```
#define BRICXX_DELEGATE_API
```

Definition at line 58 of file delegate.hpp.

7.2 delegate.hpp

Go to the documentation of this file.

```
00001 // MIT License: https://spdx.org/licenses/MIT.html
00002 //
00003 // Copyright (c) 2026 Cristian ANITA <cristian_anita@yahoo.com>
00004 //
00005 // Permission is hereby granted, free of charge, to any person obtaining a copy
00006 // of this software and associated documentation files (the "Software"), to deal
00007 // in the Software without restriction, including without limitation the rights
00008 // to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
00009 // copies of the Software, and to permit persons to whom the Software is
00010 // furnished to do so, subject to the following conditions:
00011 //
00012 // The above copyright notice and this permission notice shall be included in all
00013 // copies or substantial portions of the Software.
00014 //
00015 // THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
00016 // EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
00017 // MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.
00018 // IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM,
00019 // DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR
00020 // OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE
00021 // OR OTHER DEALINGS IN THE SOFTWARE.
00022
00023
00024 #ifndef BRICXX_DELEGATE_HPP_INCLUDED_
00025 #define BRICXX_DELEGATE_HPP_INCLUDED_
00026
00027
00028 // /Zc:__cplusplus; please see:
00029 // - MSVC now correctly reports __cplusplus:
      https://devblogs.microsoft.com/cppblog/msvc-now-correctly-reports-__cplusplus
00030 // - /Zc:__cplusplus (Enable updated __cplusplus macro):
      https://learn.microsoft.com/en-us/cpp/build/reference/zc-cplusplus
00031 #ifdef _MSC_VER
00032 #   if (_MSC_VER < 1914)
00033 #       error "msvc >= 2017 v15.7 required!!! (for /Zc:__cplusplus)"
00034 #   endif
00035 #endif
00036
00037 #if (__cplusplus < 201703L)
00038 #   error "C++ >= 17 required!!!"
00039 #endif
00040
00041
00042 #ifdef _MSC_VER
00043 #pragma warning(push)
00044 // C4646: function declared with __declspec(noreturn) has non-void return type.
00045 // DelegateBase::NullStub() has (possibly non-void) return type (it must conform to stub signature)
00046 // but it actually always throws BadDelegateCall.
00047 #pragma warning(disable: 4646)
00048 #endif
00049
00050
00051 #include <exception>
00052 #include <type_traits>
00053 #include <utility>
00054 #include <memory>
00055
00056
00057 // TODO: If Delegate is used in a shared library define BRICXX_DELEGATE_API to that library
      export/import macro.
00058 #define BRICXX_DELEGATE_API
00059
00060
00061 namespace bricxx {
00062
```

```
00063
00064
      //*****************************************************************************************************************
00163 template <typename TSignature>
00164 class Delegate;
00165
00166
00167
00168
00169
      //*****************************************************************************************************************
00171 class BRICXX_DELEGATE_API BadDelegateCall final: public ::std::exception {
00172 public:
00173     const char* what() const noexcept override;
00174 };
00175
00176
00177
00178
00179
      //*****************************************************************************************************************
00181 template <typename TRetVal, typename... TParams>
00182 class DelegateBase {
00183 public:
00184     [[nodiscard]] constexpr bool IsBinded() const noexcept;
00185     [[nodiscard]] explicit constexpr operator bool() const noexcept;
00186
00187     constexpr void Unbind() noexcept;
00188     constexpr void Reset() noexcept;
00189 protected:
00190     using ErasedObjectType = void;
00191     using StubFnPtr = TRetVal (*)(ErasedObjectType* pTypeErasedObject, TParams... params);
00192
00193
00196     constexpr DelegateBase() noexcept;
00197     constexpr DelegateBase(const DelegateBase& other) noexcept;
00199
00200     ~DelegateBase() noexcept;
00201
00202     constexpr DelegateBase& operator=(const DelegateBase& other) noexcept;
00203
00204
00207     [[nodiscard]] constexpr bool operator==(const DelegateBase& other) const noexcept;
00208     [[nodiscard]] constexpr bool operator!=(const DelegateBase& other) const noexcept;
00210
00211
00213     [[noreturn]] static TRetVal NullStub([[maybe_unused]] ErasedObjectType* pTypeErasedObject,
00214         [[maybe_unused]] TParams... params);
00215
00216
00217     StubFnPtr pStub_ = &NullStub;
00218     ErasedObjectType* pTypeErasedObject_ = nullptr;
00219 };
00220
00221
00222
00223
00224
      //*****************************************************************************************************************
00226 template <typename TRetVal, typename... TParams>
00227 class Delegate<TRetVal (TParams...)> final: private DelegateBase<TRetVal, TParams...> {
00228 public:
00229     using DelegateBase<TRetVal, TParams...>::IsBinded;
00230     using DelegateBase<TRetVal, TParams...>::operator bool;
00231
00232     using DelegateBase<TRetVal, TParams...>::Unbind;
00233     using DelegateBase<TRetVal, TParams...>::Reset;
00234
00235
00238     constexpr Delegate() noexcept;
00239     constexpr Delegate(const Delegate& other) noexcept;
00241
00242     ~Delegate() noexcept;
00243
00244     constexpr Delegate& operator=(const Delegate& other) noexcept;
00245
00246
00249     [[nodiscard]] constexpr bool operator==(const Delegate& other) const noexcept;
00250     [[nodiscard]] constexpr bool operator!=(const Delegate& other) const noexcept;
00252
00253
00257
00260     template <auto TFunction,
00261         typename = ::std::enable_if_t<
00262             ::std::is_invocable_r_v<TRetVal, decltype(TFunction), TParams...>
00263         >
00264     >
```

```
    constexpr void Bind() noexcept;

    template <class TFunctor,
        typename = ::std::enable_if_t<
            ::std::is_class_v<TFunctor>
            && ::std::is_invocable_r_v<TRetVal, TFunctor&, TParams...>
        >
    >
    constexpr void Bind(TFunctor& functor) noexcept;

    template <auto TMethod, class TClass,
        typename = ::std::enable_if_t<
            ::std::is_invocable_r_v<TRetVal, decltype(TMethod), TClass&, TParams...>
        >
    >
    constexpr void Bind(TClass& object) noexcept;



    template <auto TFunction,
        typename = ::std::enable_if_t<
            ::std::is_invocable_r_v<TRetVal, decltype(TFunction), TParams...>
        >
    >
    [[nodiscard]] constexpr bool IsBindedTo() const noexcept;

    template <class TFunctor,
        typename = ::std::enable_if_t<
            ::std::is_class_v<TFunctor>
            && ::std::is_invocable_r_v<TRetVal, TFunctor&, TParams...>
        >
    >
    [[nodiscard]] constexpr bool IsBindedTo(TFunctor& functor) const noexcept;

    template <auto TMethod, class TClass,
        typename = ::std::enable_if_t<
            ::std::is_invocable_r_v<TRetVal, decltype(TMethod), TClass&, TParams...>
        >
    >
    [[nodiscard]] constexpr bool IsBindedTo(TClass& object) const noexcept;



    template <typename... UParams,
        typename = ::std::enable_if_t<
            ::std::is_invocable_v<TRetVal (TParams...), UParams...>
        >
    >
    [[nodiscard]] TRetVal operator()(UParams&&... params);
private:
    using typename DelegateBase<TRetVal, TParams...>::ErasedObjectType;
    using typename DelegateBase<TRetVal, TParams...>::StubFnPtr;
};




//*****************************************************************************************************************
template <typename TRetVal, typename... TParams>
class Delegate<TRetVal (TParams...) const> final: private DelegateBase<TRetVal, TParams...> {
public:
    using DelegateBase<TRetVal, TParams...>::IsBinded;
    using DelegateBase<TRetVal, TParams...>::operator bool;

    using DelegateBase<TRetVal, TParams...>::Unbind;
    using DelegateBase<TRetVal, TParams...>::Reset;


    constexpr Delegate() noexcept;
    constexpr Delegate(const Delegate& other) noexcept;

    ~Delegate() noexcept;

    constexpr Delegate& operator=(const Delegate& other) noexcept;


    [[nodiscard]] constexpr bool operator==(const Delegate& other) const noexcept;
    [[nodiscard]] constexpr bool operator!=(const Delegate& other) const noexcept;



    template <class TFunctor,
        typename = ::std::enable_if_t<
            ::std::is_class_v<TFunctor>
            && ::std::is_invocable_r_v<TRetVal, const TFunctor&, TParams...>
```

```
        >
    >
    constexpr void Bind(const TFunctor& functor) noexcept;

    template <auto TMethod, class TClass,
        typename = ::std::enable_if_t<
            ::std::is_invocable_r_v<TRetVal, decltype(TMethod), const TClass&, TParams...>
        >
    >
    constexpr void Bind(const TClass& object) noexcept;




    template <class TFunctor,
        typename = ::std::enable_if_t<
            ::std::is_class_v<TFunctor>
            && ::std::is_invocable_r_v<TRetVal, const TFunctor&, TParams...>
        >
    >
    [[nodiscard]] constexpr bool IsBindedTo(const TFunctor& functor) const noexcept;

    template <auto TMethod, class TClass,
        typename = ::std::enable_if_t<
            ::std::is_invocable_r_v<TRetVal, decltype(TMethod), const TClass&, TParams...>
        >
    >
    [[nodiscard]] constexpr bool IsBindedTo(const TClass& object) const noexcept;



    template <typename... UParams,
        typename = ::std::enable_if_t<
            ::std::is_invocable_v<TRetVal (TParams...), UParams...>
        >
    >
    [[nodiscard]] TRetVal operator()(UParams&&... params) const;
private:
    using typename DelegateBase<TRetVal, TParams...>::ErasedObjectType;
    using typename DelegateBase<TRetVal, TParams...>::StubFnPtr;
};




    //*****************************************************************************************************************
template <typename TRetVal, typename... TParams>
class Delegate<TRetVal (TParams...) noexcept> final: private DelegateBase<TRetVal, TParams...> {
public:
    using DelegateBase<TRetVal, TParams...>::IsBinded;
    using DelegateBase<TRetVal, TParams...>::operator bool;

    using DelegateBase<TRetVal, TParams...>::Unbind;
    using DelegateBase<TRetVal, TParams...>::Reset;


    constexpr Delegate() noexcept;
    constexpr Delegate(const Delegate& other) noexcept;

    ~Delegate() noexcept;

    constexpr Delegate& operator=(const Delegate& other) noexcept;


    [[nodiscard]] constexpr bool operator==(const Delegate& other) const noexcept;
    [[nodiscard]] constexpr bool operator!=(const Delegate& other) const noexcept;



    template <auto TFunction,
        typename = ::std::enable_if_t<
            ::std::is_nothrow_invocable_r_v<TRetVal, decltype(TFunction), TParams...>
        >
    >
    constexpr void Bind() noexcept;

    template <class TFunctor,
        typename = ::std::enable_if_t<
            ::std::is_class_v<TFunctor>
            && ::std::is_nothrow_invocable_r_v<TRetVal, TFunctor&, TParams...>
        >
    >
    constexpr void Bind(TFunctor& functor) noexcept;

    template <auto TMethod, class TClass,
        typename = ::std::enable_if_t<
```

```
            ::std::is_nothrow_invocable_r_v<TRetVal, decltype(TMethod), TClass&, TParams...>
        >
    >
    constexpr void Bind(TClass& object) noexcept;



    template <auto TFunction,
        typename = ::std::enable_if_t<
            ::std::is_nothrow_invocable_r_v<TRetVal, decltype(TFunction), TParams...>
        >
    >
    [[nodiscard]] constexpr bool IsBindedTo() const noexcept;

    template <class TFunctor,
        typename = ::std::enable_if_t<
            ::std::is_class_v<TFunctor>
            && ::std::is_nothrow_invocable_r_v<TRetVal, TFunctor&, TParams...>
        >
    >
    [[nodiscard]] constexpr bool IsBindedTo(TFunctor& functor) const noexcept;

    template <auto TMethod, class TClass,
        typename = ::std::enable_if_t<
            ::std::is_nothrow_invocable_r_v<TRetVal, decltype(TMethod), TClass&, TParams...>
        >
    >
    [[nodiscard]] constexpr bool IsBindedTo(TClass& object) const noexcept;



    template <typename... UParams,
        typename = ::std::enable_if_t<
            ::std::is_nothrow_invocable_v<TRetVal (TParams...) noexcept, UParams...>
        >
    >
    [[nodiscard]] TRetVal operator()(UParams&&... params) noexcept;
private:
    using typename DelegateBase<TRetVal, TParams...>::ErasedObjectType;
    using typename DelegateBase<TRetVal, TParams...>::StubFnPtr;
};




//*****************************************************************************************************************
template <typename TRetVal, typename... TParams>
class Delegate<TRetVal (TParams...) const noexcept> final: private DelegateBase<TRetVal, TParams...> {
public:
    using DelegateBase<TRetVal, TParams...>::IsBinded;
    using DelegateBase<TRetVal, TParams...>::operator bool;

    using DelegateBase<TRetVal, TParams...>::Unbind;
    using DelegateBase<TRetVal, TParams...>::Reset;


    constexpr Delegate() noexcept;
    constexpr Delegate(const Delegate& other) noexcept;

    ~Delegate() noexcept;

    constexpr Delegate& operator=(const Delegate& other) noexcept;


    [[nodiscard]] constexpr bool operator==(const Delegate& other) const noexcept;
    [[nodiscard]] constexpr bool operator!=(const Delegate& other) const noexcept;



    template <class TFunctor,
        typename = ::std::enable_if_t<
            ::std::is_class_v<TFunctor>
            && ::std::is_nothrow_invocable_r_v<TRetVal, const TFunctor&, TParams...>
        >
    >
    constexpr void Bind(const TFunctor& functor) noexcept;

    template <auto TMethod, class TClass,
        typename = ::std::enable_if_t<
            ::std::is_nothrow_invocable_r_v<TRetVal, decltype(TMethod), const TClass&, TParams...>
        >
    >
    constexpr void Bind(const TClass& object) noexcept;


```

```
    template <class TFunctor,
        typename = ::std::enable_if_t<
            ::std::is_class_v<TFunctor>
            && ::std::is_nothrow_invocable_r_v<TRetVal, const TFunctor&, TParams...>
        >
    >
    [[nodiscard]] constexpr bool IsBindedTo(const TFunctor& functor) const noexcept;

    template <auto TMethod, class TClass,
        typename = ::std::enable_if_t<
            ::std::is_nothrow_invocable_r_v<TRetVal, decltype(TMethod), const TClass&, TParams...>
        >
    >
    [[nodiscard]] constexpr bool IsBindedTo(const TClass& object) const noexcept;



    template <typename... UParams,
        typename = ::std::enable_if_t<
            ::std::is_nothrow_invocable_v<TRetVal (TParams...) noexcept, UParams...>
        >
    >
    [[nodiscard]] TRetVal operator()(UParams&&... params) const noexcept;
private:
    using typename DelegateBase<TRetVal, TParams...>::ErasedObjectType;
    using typename DelegateBase<TRetVal, TParams...>::StubFnPtr;
};



//*****************************************************************************************************************
// Inline functions / template implementation

inline const char* BadDelegateCall::what() const noexcept {
    return "Bad delegate call";
}



template <typename TRetVal, typename... TParams>
inline constexpr bool DelegateBase<TRetVal, TParams...>::IsBinded() const noexcept {
    return pStub_ != &NullStub;
}

template <typename TRetVal, typename... TParams>
inline constexpr DelegateBase<TRetVal, TParams...>::operator bool() const noexcept {
    return IsBinded();
}


template <typename TRetVal, typename... TParams>
inline constexpr void DelegateBase<TRetVal, TParams...>::Unbind() noexcept {
    pStub_ = &NullStub;
    pTypeErasedObject_ = nullptr;
}

template <typename TRetVal, typename... TParams>
inline constexpr void DelegateBase<TRetVal, TParams...>::Reset() noexcept {
    Unbind();
}


template <typename TRetVal, typename... TParams>
inline constexpr DelegateBase<TRetVal, TParams...>::DelegateBase() noexcept = default;

template <typename TRetVal, typename... TParams>
inline constexpr DelegateBase<TRetVal, TParams...>::DelegateBase(const DelegateBase& other) noexcept = default;


template <typename TRetVal, typename... TParams>
inline DelegateBase<TRetVal, TParams...>::~DelegateBase() noexcept = default;


template <typename TRetVal, typename... TParams>
inline constexpr DelegateBase<TRetVal, TParams...>& DelegateBase<TRetVal, TParams...>::operator=(
        const DelegateBase& other) noexcept = default;


template <typename TRetVal, typename... TParams>
```

```
inline constexpr bool DelegateBase<TRetVal, TParams...>::operator==(const DelegateBase& other) const noexcept {
    return (pStub_ == other.pStub_) && (pTypeErasedObject_ == other.pTypeErasedObject_);
}

template <typename TRetVal, typename... TParams>
inline constexpr bool DelegateBase<TRetVal, TParams...>::operator!=(const DelegateBase& other) const noexcept {
    return !(*this == other);
}


template <typename TRetVal, typename... TParams>
TRetVal DelegateBase<TRetVal, TParams...>::NullStub([[maybe_unused]] ErasedObjectType* pTypeErasedObject,
        [[maybe_unused]] TParams... params) {
    throw BadDelegateCall();
}




template <typename TRetVal, typename... TParams>
inline constexpr Delegate<TRetVal (TParams...)>::Delegate() noexcept = default;

template <typename TRetVal, typename... TParams>
inline constexpr Delegate<TRetVal (TParams...)>::Delegate(const Delegate& other) noexcept = default;


template <typename TRetVal, typename... TParams>
inline Delegate<TRetVal (TParams...)>::~Delegate() noexcept = default;


template <typename TRetVal, typename... TParams>
inline constexpr Delegate<TRetVal (TParams...)>& Delegate<TRetVal (TParams...)>::operator=(const Delegate& other)
        noexcept = default;


template <typename TRetVal, typename... TParams>
inline constexpr bool Delegate<TRetVal (TParams...)>::operator==(const Delegate& other) const noexcept {
    return DelegateBase<TRetVal, TParams...>::operator==(other);
}

template <typename TRetVal, typename... TParams>
inline constexpr bool Delegate<TRetVal (TParams...)>::operator!=(const Delegate& other) const noexcept {
    return !(*this == other);
}


template <typename TRetVal, typename... TParams>
template <auto TFunction, typename>
inline constexpr void Delegate<TRetVal (TParams...)>::Bind() noexcept {
    this->pStub_ = static_cast<StubFnPtr>(
        // FunctionStub()
        []([[maybe_unused]] ErasedObjectType* pTypeErasedObject, TParams... params) -> TRetVal {
            return (*TFunction)(::std::forward<TParams>(params)...);
        }
    );
    this->pTypeErasedObject_ = nullptr;
}

template <typename TRetVal, typename... TParams>
template <class TFunctor, typename>
inline constexpr void Delegate<TRetVal (TParams...)>::Bind(TFunctor& functor) noexcept {
    this->pStub_ = static_cast<StubFnPtr>(
        // FunctorStub()
        [](ErasedObjectType* pTypeErasedObject, TParams... params) -> TRetVal {
            return (*static_cast<TFunctor*>(pTypeErasedObject))(::std::forward<TParams>(params)...);
        }
    );
    this->pTypeErasedObject_ = ::std::addressof(functor);
}

template <typename TRetVal, typename... TParams>
template <auto TMethod, class TClass, typename>
inline constexpr void Delegate<TRetVal (TParams...)>::Bind(TClass& object) noexcept {
    static_assert(sizeof(TClass*) == sizeof(ErasedObjectType*));

    this->pStub_ = static_cast<StubFnPtr>(
        // MethodStub()
        [](ErasedObjectType* pTypeErasedObject, TParams... params) -> TRetVal {
            return (static_cast<TClass*>(pTypeErasedObject)->*TMethod)(::std::forward<TParams>(params)...);
        }
```

```
    );
    this->pTypeErasedObject_ = ::std::addressof(object);
}


template <typename TRetVal, typename... TParams>
template <auto TFunction, typename>
inline constexpr bool Delegate<TRetVal (TParams...)>::IsBindedTo() const noexcept {
    Delegate delegate;
    delegate.Bind<TFunction>();
    return *this == delegate;
}

template <typename TRetVal, typename... TParams>
template <class TFunctor, typename>
inline constexpr bool Delegate<TRetVal (TParams...)>::IsBindedTo(TFunctor& functor) const noexcept {
    Delegate delegate;
    delegate.Bind(functor);
    return *this == delegate;
}

template <typename TRetVal, typename... TParams>
template <auto TMethod, class TClass, typename>
inline constexpr bool Delegate<TRetVal (TParams...)>::IsBindedTo(TClass& object) const noexcept {
    Delegate delegate;
    delegate.Bind<TMethod>(object);
    return *this == delegate;
}


template <typename TRetVal, typename... TParams>
template <typename... UParams, typename>
inline TRetVal Delegate<TRetVal (TParams...)>::operator()(UParams&&... params) {
    return (*this->pStub_)(this->pTypeErasedObject_, ::std::forward<UParams>(params)...);
}




template <typename TRetVal, typename... TParams>
inline constexpr Delegate<TRetVal (TParams...) const>::Delegate() noexcept = default;

template <typename TRetVal, typename... TParams>
inline constexpr Delegate<TRetVal (TParams...) const>::Delegate(const Delegate& other) noexcept = default;


template <typename TRetVal, typename... TParams>
inline Delegate<TRetVal (TParams...) const>::~Delegate() noexcept = default;


template <typename TRetVal, typename... TParams>
inline constexpr Delegate<TRetVal (TParams...) const>& Delegate<TRetVal (TParams...) const>::operator=(const Delegate& other)
        noexcept = default;


template <typename TRetVal, typename... TParams>
inline constexpr bool Delegate<TRetVal (TParams...) const>::operator==(const Delegate& other) const noexcept {
    return DelegateBase<TRetVal, TParams...>::operator==(other);
}

template <typename TRetVal, typename... TParams>
inline constexpr bool Delegate<TRetVal (TParams...) const>::operator!=(const Delegate& other) const noexcept {
    return !(*this == other);
}


template <typename TRetVal, typename... TParams>
template <class TFunctor, typename>
inline constexpr void Delegate<TRetVal (TParams...) const>::Bind(const TFunctor& functor) noexcept {
    this->pStub_ = static_cast<StubFnPtr>(
        // FunctorStub()
        [](ErasedObjectType* pTypeErasedObject, TParams... params) -> TRetVal {
            // restoring removed const
            return (*static_cast<const TFunctor*>(pTypeErasedObject))(::std::forward<TParams>(params)...);
        }
    );

    // removed const will be restored in FunctorStub()
    this->pTypeErasedObject_ = const_cast<ErasedObjectType*>(static_cast<const ErasedObjectType*>(::std::addressof(functor)));
}

```

```
template <typename TRetVal, typename... TParams>
template <auto TMethod, class TClass, typename>
inline constexpr void Delegate<TRetVal (TParams...) const>::Bind(const TClass& object) noexcept {
    static_assert(sizeof(TClass*) == sizeof(ErasedObjectType*));

    this->pStub_ = static_cast<StubFnPtr>(
        // MethodStub()
        [](ErasedObjectType* pTypeErasedObject, TParams... params) -> TRetVal {
            // restoring removed const
            return (static_cast<const TClass*>(pTypeErasedObject)->*TMethod)(::std::forward<TParams>(params)...);
        }
    );

    // removed const will be restored in MethodStub()
    this->pTypeErasedObject_ = const_cast<ErasedObjectType*>(static_cast<const ErasedObjectType*>(::std::addressof(object)));
}


template <typename TRetVal, typename... TParams>
template <class TFunctor, typename>
inline constexpr bool Delegate<TRetVal (TParams...) const>::IsBindedTo(const TFunctor& functor) const noexcept {
    Delegate delegate;
    delegate.Bind(functor);
    return *this == delegate;
}

template <typename TRetVal, typename... TParams>
template <auto TMethod, class TClass, typename>
inline constexpr bool Delegate<TRetVal (TParams...) const>::IsBindedTo(const TClass& object) const noexcept {
    Delegate delegate;
    delegate.Bind<TMethod>(object);
    return *this == delegate;
}


template <typename TRetVal, typename... TParams>
template <typename... UParams, typename>
inline TRetVal Delegate<TRetVal (TParams...) const>::operator()(UParams&&... params) const {
    return (*this->pStub_)(this->pTypeErasedObject_, ::std::forward<UParams>(params)...);
}




template <typename TRetVal, typename... TParams>
inline constexpr Delegate<TRetVal (TParams...) noexcept>::Delegate() noexcept = default;

template <typename TRetVal, typename... TParams>
inline constexpr Delegate<TRetVal (TParams...) noexcept>::Delegate(const Delegate& other) noexcept = default;


template <typename TRetVal, typename... TParams>
inline Delegate<TRetVal (TParams...) noexcept>::~Delegate() noexcept = default;


template <typename TRetVal, typename... TParams>
inline constexpr Delegate<TRetVal (TParams...) noexcept>& Delegate<TRetVal (TParams...) noexcept>::operator=(
        const Delegate& other) noexcept = default;


template <typename TRetVal, typename... TParams>
inline constexpr bool Delegate<TRetVal (TParams...) noexcept>::operator==(const Delegate& other) const noexcept {
    return DelegateBase<TRetVal, TParams...>::operator==(other);
}

template <typename TRetVal, typename... TParams>
inline constexpr bool Delegate<TRetVal (TParams...) noexcept>::operator!=(const Delegate& other) const noexcept {
    return !(*this == other);
}


template <typename TRetVal, typename... TParams>
template <auto TFunction, typename>
inline constexpr void Delegate<TRetVal (TParams...) noexcept>::Bind() noexcept {
    this->pStub_ = static_cast<StubFnPtr>(
        // FunctionStub()
        []([[maybe_unused]] ErasedObjectType* pTypeErasedObject, TParams... params) noexcept -> TRetVal {
            return (*TFunction)(::std::forward<TParams>(params)...);
```

```
        }
    );
    this->pTypeErasedObject_ = nullptr;
}

template <typename TRetVal, typename... TParams>
template <class TFunctor, typename>
inline constexpr void Delegate<TRetVal (TParams...) noexcept>::Bind(TFunctor& functor) noexcept {
    this->pStub_ = static_cast<StubFnPtr>(
        // FunctorStub()
        [](ErasedObjectType* pTypeErasedObject, TParams... params) noexcept -> TRetVal {
            return (*static_cast<TFunctor*>(pTypeErasedObject))(::std::forward<TParams>(params)...);
        }
    );
    this->pTypeErasedObject_ = ::std::addressof(functor);
}

template <typename TRetVal, typename... TParams>
template <auto TMethod, class TClass, typename>
inline constexpr void Delegate<TRetVal (TParams...) noexcept>::Bind(TClass& object) noexcept {
    static_assert(sizeof(TClass*) == sizeof(ErasedObjectType*));

    this->pStub_ = static_cast<StubFnPtr>(
        // MethodStub()
        [](ErasedObjectType* pTypeErasedObject, TParams... params) noexcept -> TRetVal {
            return
(static_cast<TClass*>(pTypeErasedObject)->*TMethod)(::std::forward<TParams>(params)...);
        }
    );
    this->pTypeErasedObject_ = ::std::addressof(object);
}


template <typename TRetVal, typename... TParams>
template <auto TFunction, typename>
inline constexpr bool Delegate<TRetVal (TParams...) noexcept>::IsBindedTo() const noexcept {
    Delegate delegate;
    delegate.Bind<TFunction>();
    return *this == delegate;
}

template <typename TRetVal, typename... TParams>
template <class TFunctor, typename>
inline constexpr bool Delegate<TRetVal (TParams...) noexcept>::IsBindedTo(TFunctor& functor) const
noexcept {
    Delegate delegate;
    delegate.Bind(functor);
    return *this == delegate;
}

template <typename TRetVal, typename... TParams>
template <auto TMethod, class TClass, typename>
inline constexpr bool Delegate<TRetVal (TParams...) noexcept>::IsBindedTo(TClass& object) const
noexcept {
    Delegate delegate;
    delegate.Bind<TMethod>(object);
    return *this == delegate;
}


template <typename TRetVal, typename... TParams>
template <typename... UParams, typename>
inline TRetVal Delegate<TRetVal (TParams...) noexcept>::operator()(UParams&&... params) noexcept {
    return (*this->pStub_)(this->pTypeErasedObject_, ::std::forward<UParams>(params)...);
}




template <typename TRetVal, typename... TParams>
inline constexpr Delegate<TRetVal (TParams...) const noexcept>::Delegate() noexcept = default;

template <typename TRetVal, typename... TParams>
inline constexpr Delegate<TRetVal (TParams...) const noexcept>::Delegate(const Delegate& other)
noexcept = default;


template <typename TRetVal, typename... TParams>
inline Delegate<TRetVal (TParams...) const noexcept>::~Delegate() noexcept = default;


template <typename TRetVal, typename... TParams>
inline constexpr Delegate<TRetVal (TParams...) const noexcept>& Delegate<TRetVal (TParams...) const
noexcept>::operator=(
        const Delegate& other) noexcept = default;


```

```
template <typename TRetVal, typename... TParams>
inline constexpr bool Delegate<TRetVal (TParams...) const noexcept>::operator==(const Delegate& other) const noexcept {
    return DelegateBase<TRetVal, TParams...>::operator==(other);
}

template <typename TRetVal, typename... TParams>
inline constexpr bool Delegate<TRetVal (TParams...) const noexcept>::operator!=(const Delegate& other) const noexcept {
    return !(*this == other);
}


template <typename TRetVal, typename... TParams>
template <class TFunctor, typename>
inline constexpr void Delegate<TRetVal (TParams...) const noexcept>::Bind(const TFunctor& functor) noexcept {
    this->pStub_ = static_cast<StubFnPtr>(
        // FunctorStub()
        [](ErasedObjectType* pTypeErasedObject, TParams... params) noexcept -> TRetVal {
            // restoring removed const
            return (*static_cast<const TFunctor*>(pTypeErasedObject))(::std::forward<TParams>(params)...);
        }
    );

    // removed const will be restored in FunctorStub()
    this->pTypeErasedObject_ = const_cast<ErasedObjectType*>(static_cast<const ErasedObjectType*>(::std::addressof(functor)));
}

template <typename TRetVal, typename... TParams>
template <auto TMethod, class TClass, typename>
inline constexpr void Delegate<TRetVal (TParams...) const noexcept>::Bind(const TClass& object) noexcept {
    static_assert(sizeof(TClass*) == sizeof(ErasedObjectType*));

    this->pStub_ = static_cast<StubFnPtr>(
        // MethodStub()
        [](ErasedObjectType* pTypeErasedObject, TParams... params) noexcept -> TRetVal {
            // restoring removed const
            return (static_cast<const TClass*>(pTypeErasedObject)->*TMethod)(::std::forward<TParams>(params)...);
        }
    );

    // removed const will be restored in MethodStub()
    this->pTypeErasedObject_ = const_cast<ErasedObjectType*>(static_cast<const ErasedObjectType*>(::std::addressof(object)));
}


template <typename TRetVal, typename... TParams>
template <class TFunctor, typename>
inline constexpr bool Delegate<TRetVal (TParams...) const noexcept>::IsBindedTo(const TFunctor& functor) const noexcept {
    Delegate delegate;
    delegate.Bind(functor);
    return *this == delegate;
}

template <typename TRetVal, typename... TParams>
template <auto TMethod, class TClass, typename>
inline constexpr bool Delegate<TRetVal (TParams...) const noexcept>::IsBindedTo(const TClass& object) const noexcept {
    Delegate delegate;
    delegate.Bind<TMethod>(object);
    return *this == delegate;
}


template <typename TRetVal, typename... TParams>
template <typename... UParams, typename>
inline TRetVal Delegate<TRetVal (TParams...) const noexcept>::operator()(UParams&&... params) const noexcept {
    return (*this->pStub_)(this->pTypeErasedObject_, ::std::forward<UParams>(params)...);
}




}   // namespace bricxx


#ifdef _MSC_VER
#pragma warning(pop)
#endif
```

```
#endif  // BRICXX_DELEGATE_HPP_INCLUDED_
```

Index

~Delegate
- bricxx::Delegate< TRetVal(TParams...) const >, 15
- bricxx::Delegate< TRetVal(TParams...) const noexcept >, 19
- bricxx::Delegate< TRetVal(TParams...) noexcept >, 22
- bricxx::Delegate< TRetVal(TParams...)>, 26

~DelegateBase
- bricxx::DelegateBase< TRetVal, TParams >, 30

Bind
- bricxx::Delegate< TRetVal(TParams...) const >, 16
- bricxx::Delegate< TRetVal(TParams...) const noexcept >, 19
- bricxx::Delegate< TRetVal(TParams...) noexcept >, 23
- bricxx::Delegate< TRetVal(TParams...)>, 27

bricxx, 9

bricxx::BadDelegateCall, 11
- what, 11

bricxx::Delegate< TRetVal(TParams...) const >, 14
- ~Delegate, 15
- Bind, 16
- Delegate, 15
- IsBindedTo, 16
- operator!=, 15
- operator(), 16
- operator=, 15
- operator==, 15

bricxx::Delegate< TRetVal(TParams...) const noexcept >, 17
- ~Delegate, 19
- Bind, 19
- Delegate, 18
- IsBindedTo, 20
- operator!=, 19
- operator(), 20
- operator=, 19
- operator==, 19

bricxx::Delegate< TRetVal(TParams...) noexcept >, 21
- ~Delegate, 22
- Bind, 23
- Delegate, 22
- IsBindedTo, 23, 24
- operator!=, 22
- operator(), 24
- operator=, 22
- operator==, 22

bricxx::Delegate< TRetVal(TParams...)>, 25
- ~Delegate, 26
- Bind, 27
- Delegate, 26
- IsBindedTo, 27, 28
- operator!=, 26
- operator(), 28
- operator=, 26
- operator==, 26

bricxx::Delegate< TSignature >, 12

bricxx::DelegateBase< TRetVal, TParams >, 29
- ~DelegateBase, 30
- DelegateBase, 30
- ErasedObjectType, 30
- IsBinded, 31
- NullStub, 31
- operator bool, 31
- operator!=, 31
- operator=, 31
- operator==, 31
- pStub_, 32
- pTypeErasedObject_, 32
- Reset, 31
- StubFnPtr, 30
- Unbind, 31

BRICXX_DELEGATE_API
- delegate.hpp, 34

Delegate
- bricxx::Delegate< TRetVal(TParams...) const >, 15
- bricxx::Delegate< TRetVal(TParams...) const noexcept >, 18
- bricxx::Delegate< TRetVal(TParams...) noexcept >, 22
- bricxx::Delegate< TRetVal(TParams...)>, 26

delegate.hpp, 33
- BRICXX_DELEGATE_API, 34

DelegateBase
- bricxx::DelegateBase< TRetVal, TParams >, 30

ErasedObjectType
- bricxx::DelegateBase< TRetVal, TParams >, 30

IsBinded
- bricxx::DelegateBase< TRetVal, TParams >, 31

IsBindedTo
- bricxx::Delegate< TRetVal(TParams...) const >, 16

bricxx::Delegate< TRetVal(TParams...) const noexcept >, 20
bricxx::Delegate< TRetVal(TParams...) noexcept >, 23, 24
bricxx::Delegate< TRetVal(TParams...)>, 27, 28

NullStub
bricxx::DelegateBase< TRetVal, TParams >, 31

operator bool
bricxx::DelegateBase< TRetVal, TParams >, 31
operator!=
bricxx::Delegate< TRetVal(TParams...) const >, 15
bricxx::Delegate< TRetVal(TParams...) const noexcept >, 19
bricxx::Delegate< TRetVal(TParams...) noexcept >, 22
bricxx::Delegate< TRetVal(TParams...)>, 26
bricxx::DelegateBase< TRetVal, TParams >, 31
operator()
bricxx::Delegate< TRetVal(TParams...) const >, 16
bricxx::Delegate< TRetVal(TParams...) const noexcept >, 20
bricxx::Delegate< TRetVal(TParams...) noexcept >, 24
bricxx::Delegate< TRetVal(TParams...)>, 28
operator=
bricxx::Delegate< TRetVal(TParams...) const >, 15
bricxx::Delegate< TRetVal(TParams...) const noexcept >, 19
bricxx::Delegate< TRetVal(TParams...) noexcept >, 22
bricxx::Delegate< TRetVal(TParams...)>, 26
bricxx::DelegateBase< TRetVal, TParams >, 31
operator==
bricxx::Delegate< TRetVal(TParams...) const >, 15
bricxx::Delegate< TRetVal(TParams...) const noexcept >, 19
bricxx::Delegate< TRetVal(TParams...) noexcept >, 22
bricxx::Delegate< TRetVal(TParams...)>, 26
bricxx::DelegateBase< TRetVal, TParams >, 31

pStub_
bricxx::DelegateBase< TRetVal, TParams >, 32
pTypeErasedObject_
bricxx::DelegateBase< TRetVal, TParams >, 32

Reset
bricxx::DelegateBase< TRetVal, TParams >, 31

StubFnPtr
bricxx::DelegateBase< TRetVal, TParams >, 30

Unbind
bricxx::DelegateBase< TRetVal, TParams >, 31

what
bricxx::BadDelegateCall, 11